

ЛЕКЦИЯ 11 КОГЕРЕНТНОСТЬ МНОГОУРОВНЕВОЙ ПАМЯТИ

Модели состоятельности памяти

Одной из основных проблем масштабируемых многопроцессорных вычислительных систем является построение распределенной разделяемой памяти, имеющей единое адресное пространство и доступ к объектам через операции чтения и записи.

Важнейшая задача при этом состоит в том, как извещать процессоры об изменениях, вызванных выполнением команд записи. Необходимость увеличения быстродействия вычислителей приводит к применению многоуровневой памяти и механизмов отложенной модификации удаленных копий одного и того же экземпляра данных, находящегося в кэшах различных процессоров и оперативной памяти.

При реализации изменений удаленных копий применяется **коммуникационный протокол**, обеспечивающий доставку изменений, и **протокол согласованности** или **когерентности** состояния памяти, предотвращающий использование неизмененных копий модифицированного в одном из процессоров элемента данных.

Для сохранения когерентности данных в соответствии с моделью строгой состоятельности памяти операция чтения из разделяемой памяти должна возвращать последнее записанное значение. Поэтому **при модификации одной из множества копий остальные должны быть либо изменены, либо объявлены несостоятельными** (не соответствующими последнему изменению).

Выполнение этих действий требует реализации средств быстрого распространения изменений, однако в любом случае **следование модели строгой состоятельности** (немедленного изменения значения элемента данных на всех уровнях иерархической памяти) **ведет к снижению производительности**.

Для повышения производительности применяют **модели ослабленной состоятельности памяти**, допускающие появление несогласованных копий данных в ходе их параллельной обработки с последующим обеспечением когерентности копий, при ведущем к тому же результату исполнения приложения, что и при строгой состоятельности.

Любая реализация разделяемой памяти многопроцессорных вычислительных систем характеризуется совокупностью признаков:

- **способ реализации** (аппаратный, программный, аппаратно-программный);
- **тип разделяемых объектов данных** архитектуры процессора и/или операционной системы (слово, кэш-строка, страница, сегмент, фрагмент, объект);
- **модель состоятельности памяти**, определяющая допустимые последовательности доступа в память (SRSW – "один читатель/один писатель", MRSW – "много читателей/один писатель", MRMW – "много читателей/много писателей");
- **политика обеспечения когерентности данных** (модификация, объявление несостоятельными);
- **механизм управления распределением памяти и размещением данных** (централизованный/распределенный, статический/ динамический).

Среди широко используемых в настоящее время моделей состоятельности памяти различают следующие.

1. **Строгая** (strict): каждая операция чтения возвращает последнее записанное значение.
2. **Последовательная** (sequential): все процессоры в системе соблюдают один и тот же порядок выполнения записи и чтения. Процессор, выполняющий запись, приостанавливается до получения подтверждений об объявлении несостоятельными всех копий модифицируемых данных или о модификации этих копий.
3. **Процессорная** (processor): наблюдаемый в двух процессорах порядок выполнения операций чтения-записи может не совпадать, но порядок выполнения записей, производимых каждым процессором, должен быть одним и тем же.
4. **Слабая** (weak): вводит разграничение между обычным и синхронизованным доступом в память при выполнении состоятельности только для обращений к памяти в точках синхронизации различных уровней памяти. Разрешается несогласованность данных вне точек синхронизации.
5. **Свободная** (release): уточняет модель слабой состоятельности и требует состоятельности только между парой операций синхронизации acquire - release. Операция acquire открывает критический интервал и обеспечивает исключительный доступ процессора к разделяемым данным. Операция release закрывает критический интервал и разрешает всем процессорам

доступ к разделяемым данным. При этом операции синхронизации, выполняемые в разных процессорах, должны удовлетворять модели последовательной или процессорной состоятельности памяти.

Свободная модель состоятельности памяти образует целый класс моделей. В ЭТОТ класс входят следующие модификации.

Неторопливо-свободная (lazy release): модель свободной состоятельности, при которой модификация разделяемой памяти откладывается до момента непосредственного обращения к ней. Для данной модели характерно то, что при выполнении операции acquire передаются все предшествующие результаты команд записи, которые исполнялись до наступившего момента синхронизации.

Нетерпеливо-свободная (eager release): модель свободной состоятельности с передачей изменений разделяемой памяти при выполнении операции release, независимо от того, когда эти изменения будут востребованы.

Интервально-свободная (scope): модель свободной состоятельности с разбиением исполнения на глобальные и локальные интервалы гарантирует состоятельность разделяемых данных для всех процессоров только в конце каждого глобального интервала и для последовательности локальных интервалов внутри каждого процессора.

Последовательно-свободная (entry): вариант модели свободной состоятельности, требующий доступа к разделяемым данным, защищённого синхропримитивами. Значения синхронизирующих переменных должны модифицироваться согласно модели последовательной согласованности, при которой все процессоры имеют одну и ту же последовательность обращений к каждой из этих переменных.

Неявная (аппаратная) когерентность

Современные микропроцессоры, применяемые для построения многопроцессорных вычислительных систем, имеют встроенные аппаратные средства обеспечения когерентности данных в иерархической памяти. когерентность кэшей процессоров вычислителя обеспечивается с помощью меж узловых (межмодульных) пересылок.

Размер разделяемой процессорами единицы данных обычно составляет кэш-строку. Методы реализации когерентности определяются видом памяти вычислительной системы (сосредоточенная или распределенная) и типом коммуникационной среды, объединяющей память и процессоры.

Аппаратная реализация когерентности универсальна и не требует наличия в программах дополнительного кода для организации разделяемой памяти. При этом обеспечивается минимальная задержка согласования копий данных и высокая пропускная способность всего механизма когерентности.

СОСРЕДОТОЧЕННАЯ ПАМЯТЬ

Для вычислительных систем с сосредоточенной памятью, реализуемых, как правило, в симметричных многопроцессорных архитектурах, наиболее широкое распространение получил алгоритм поддержки когерентности кэш-памяти **MESI** (Modified, Exclusive, Shared, Invalid). Этот алгоритм обеспечивает когерентность кэш-памяти с обратной записью и минимизирует пересылки данных между основной памятью и кэшем. В общем случае в SMP системах возможна как модификация данных, так и объявление модифицируемых данных несостоятельными.

Алгоритм MESI предполагает:

- вычислитель имеет общую разделяемую память,
- в каждом процессоре содержится локальная кэш-память,
- все процессоры объединены между собой и с основной памятью посредством шины.

Кроме того, к шине подключены внешние устройства. Ключевым в данной конструкции является то, что все действия, выполняемые на шине по инициативе процессоров и внешних устройств с копиями строк в кэшах процессоров и основной памяти, доступны для отслеживания всем процессорам.

При этом в каждый момент времени по шине информация передается одним источником, а воспринимают ее все подключенные к шине абоненты. Если в качестве коммуникационной среды вместо шины используется другой тип соединения, то для работоспособности алгоритма **MESI** необходимо обеспечение указанного "прослушивания" транзакций всеми процессорами вычислительной системы.

При выполнении алгоритма **MESI** каждая строка кэш-памяти процессора может находиться в одном из следующих состояний:

М – строка модифицирована (доступна по чтению и записи только в данном процессоре, поскольку модифицирована командой записи по сравнению со строкой основной памяти);

Е – строка монопольно копированная (доступна по чтению и записи в данном процессоре и в основной памяти);

S – строка множественно копированная или разделяемая (доступна по чтению и записи в данном процессоре, основной памяти и в других процессорах, содержащих ее копию);

I – строка, невозможная к использованию (не доступна ни по чтению, ни по записи).

Состояние строк используется, с одной стороны, для определения процессором возможности локального доступа к кэш-памяти (без выхода на шину), а с другой стороны, для управления механизмом когерентности.

Для управления режимом работы механизма поддержки когерентности применяется бит **WT**, содержащийся в строке КЭШа. Состояние 1 бита **WT** задает режим сквозной записи (write-through), а состояние 0 – режим обратной записи (write-back) в кэш-память.

Динамика состояний строки кэш-памяти, к которой выполняется доступ

Исходное состояние строки	Состояние после чтения	Состояние после записи
М	М	М
Е	Е	М
S	S	Сквозная запись в основную память. Если WT=1 то Е иначе S
I	Если WT=1 то Е иначе S. Обновление строки путем чтения из основной памяти	Сквозная запись в основную память, I

Промех чтения в кэш-памяти вызывает загрузку строки из основной памяти и приведение ее в состояние **Е** или **S**. Заполнение кэш-памяти происходит только при промахх чтения.

В случае промеха записи транзакция записи помещается в буфер и посылается в основную память при предоставлении шины.

При несостоятельной строке в состоянии **I** команда чтения адресуемого объекта из этой строки вызывает чтение строки из основной памяти, размещение ее в кэш-памяти и изменение ее состояния в кэше на **Е** (в режиме сквозной записи) или **S** (в режиме обратной записи).

Состояние **M** не изменяется командами чтения и записи.

В состоянии **E** чтение из строки сохраняет прежнее состояние, а запись переводит строку в состояние **M**.

В состоянии **S** чтение из строки не изменяет ее состояние. Если установлен режим сквозной записи, то после завершения записи состояние строки меняется на **E**. Если же установлен режим обратной записи, то выполняется сквозная запись в основную память, но состояние строки не изменяется.

В состоянии **I** команда записи в строку изменяет только содержимое строки основной памяти, но не изменяет содержимое кэш-памяти и сохраняет состояние строки **I**.

Для поддержки когерентности строк кэш-памяти при выполнении операций ввода-вывода и обращении в основную память других процессоров на шине генерируются специальные циклы опроса состояния кэш-памятей. В рамках этих циклов выясняется хранится ли в кэшах других процессоров строка, содержащая используемый в операции адресуемый объект. Возможен режим принудительного перевода строки в состояние **I** с помощью сигнала **INV**.

Изменение состояний строк под управлением сигнала **INV**

Исходное состояние	Измененное состояние	
	INV=0	INV=1
M	S , обратная запись строки	I , обратная запись строки
E	S	I
S	S	I
I	I	I

РАСПРЕДЕЛЕННАЯ ПАМЯТЬ

Вычислительные системы с памятью, физически распределенной по вычислительным узлам (модулям), но имеющей единое пространство адресов (единую карту памяти), применяют различные алгоритмы реализации когерентности.

Простейший из них заключается в том, что при каждом промахе в кэш:

1. В любом процессоре инициируется запрос требуемой строки из резидентного блока памяти (того блока памяти, в котором эта строка размещена).
2. Запрос передается через коммутатор или другую коммуникационную систему в узел с резидентным для строки блоком памяти.
3. Из резидентного узла необходимая строка пересылается в узел, в котором произошел промах.

В соответствии с такой процедурой обеспечивается, в том числе, начальное заполнение кэшей.

Каждый узел при этом для каждой резидентной строки ведет список узлов, в кэшах которых размещается данная разделяемая узлами вычислителя строка или организуется распределенный по узлам список этих строк.

Для обеспечения когерентности кэшей процессоров после выполнения операции записи процессор приостанавливается до завершения исполнения следующей последовательности действий:

1. Измененная строка кэша пересылается в резидентный блок памяти.
2. Разделяемая строка пересылается из резидентной памяти во все узлы, перечисленные в списке разделяющих эту строку.
3. После получения подтверждений о том, что все копии заменены, резидентный узел пересылает разрешение продолжить вычисления в процессор, приостановленный после записи.

Основным недостатком данного алгоритма является большое время простоя процессоров при выполнении операций записи в кэш. На практике применяются более сложные алгоритмы, обеспечивающие меньшие простои процессоров.

Более совершенным является рассматриваемый ниже алгоритм **DASH**. Каждый блок памяти имеет для каждой строки, резидентной в узле, список

узлов, в кэшах которых размещены копии строк. Каждая резидентная строка может находиться в трех **глобальных** состояниях:

- **NC «не кэшированная»**, если копия строки не находится в кэше какого-либо другого узла, кроме, возможно, резидентного для этой строки;
- **RS «удаленно-разделенная»**, если копии строки размещены в кэшах других узлов;
- **RM «удаленно-измененная»**, если строка изменена операцией записи в каком-либо узле.

Кроме того, каждая строка кэша может находиться в одном из трех **локальных** состояний:

- **NU «невозможная к использованию»**;
- **S «разделяемая»**, если есть неизменная копия, которая, возможно, размещается в других кэшах;
- **M «измененная»**, если копия изменена операцией записи.

ЧТЕНИЕ

Каждый процессор может читать из своего кэша лишь в локальных состояниях читаемой строки «разделяемая» или «измененная». Если строка отсутствует в кэше или находится в состоянии «невозможная к использованию», то посылается запрос «промах чтения», который направляется в узел, резидентный по отношению к требуемой строке.

При глобальном состоянии строки в резидентном узле «некэшированная» или «удаленно-разделенная» копия строки посылается в запросивший узел. Кроме того, список узлов, содержащих копии рассматриваемой строки, пополняется узлом, запросившим копию.

В глобальном состоянии строки «удаленно-измененная» запрос «промах чтения» перенаправляется в узел, содержащий измененную строку. Данный узел пересылает требуемую строку в запросивший и резидентный узлы и устанавливает в резидентном узле для этой строки состояние «удаленно-разделенная».

ЗАПИСЬ

При выполнении операции записи в строку, локальное состояние которой «измененная», вычисления продолжаются сразу же после завершения записи.

В локальных состояниях строки «невозможная к использованию» или «разделяемая» запись в нее начинается с посылки в резидентный для строки узел запроса на захват ее в исключительное использование и приостановки выполнения записи до получения подтверждений о том, что остальные узлы, разделяющие с ним рассматриваемую строку, перевели ее копии в локальное состояние «невозможная к использованию».

Если глобальное состояние строки в резидентном узле «не кэшированная», то строка отсылается запросившему узлу и этот узел продолжает приостановленные вычисления.

Если глобальное состояние строки «удаленно-разделенная», то резидентный узел рассылает по списку всем узлам, имеющим копию строки, запрос на перевод этих строк в локальное состояние "невозможная к использованию". При получении данного запроса каждый из узлов изменяет состояние своей копии строки на «невозможное к использованию» и посылает подтверждение исполнения в узел, инициировавший операцию записи, а в приостановленном резидентном узле строка после исполнения записи переходит в глобальное состояние «удаленно-измененная».

Исходное состояние		Состояние после чтения		Состояние после записи	
Глобальное	Локальное	Глобальное	Локальное	Глобальное	Локальное
NC		RS или NC при чтении в резидентном узле	S	RM/NC	M
RS	NU	RS	S	RM	M
	S				
	M				
RM	NU	RS	S	RM	M
	S				
	M				
ОЗУ	КЭШ	ОЗУ	КЭШ	ОЗУ	КЭШ

Реализация алгоритма когерентности с большим быстродействием возможна за счет учета специфики параллельных программ, в которых асинхронно используются одни и те же переменные на каждом временном интервале только одним процессором с последующим переносом обработки на другой процессор.

Такой тип вычислений позволяет организовать более эффективные схемы передачи необходимых строк из кэша одного процессора в кэш другого.

В вычислительных системах, коммуникационные среды которых основаны на коммутаторах с временным разделением или с пространственным разделением, интерфейс каждого узла с этим коммутатором «прослушивает» все передачи через коммутатор и не ведет списки узлов, разделяющих строки. В коммуникационных средах, построенных на составных распределенных коммутаторах, необходима дополнительная память для ведения и сопровождения списков разделяемых строк. Когерентность обеспечивается при этом хранением информации о состоянии и местонахождении каждой строки разделяемой основной памяти. Элемент такого списка для одной строки должен содержать указанную служебную информацию и ссылки «вперед» и «назад» по списку других строк.

Явная (программная) когерентность

Явная реализация когерентности предусматривает применение программистом специальных команд для работы с локальной памятью и управления контроллерами каналов коммуникационной среды, связывающей узлы вычислительной системы с массовым параллелизмом.

Основная задача разработчика приложения при этом – эффективное программирование параллельных процессов, совмещающих вычисления и передачу данных между узлами и минимизирующих объем передаваемой информации.

Поскольку память в узлах является многоуровневой, то очевидно, что получаемые по коммуникационному интерфейсу строки данных делают несостоятельными копии этих строк в кэш-памяти. Поэтому нужно предусмотреть организацию когерентности прибывшей и кэшированной строк.

Протоколы когерентности памяти при программной реализации являются надстройкой над аппаратными средствами передачи сообщений, что априори создает проблемы с недостаточной пропускной способностью при создании удаленных копий данных, передаче изменений и миграции данных.

Однако программная реализация позволяет учитывать особенности прикладных программ и применять широкое разнообразие протоколов согласованности данных. Различают следующие варианты поддержки когерентности:

1. При наличии дубликатов признаков (тегов) строк кэш-памяти в контроллере прямого доступа делать по получении строки несостоятельной только действительно необходимую строку.
2. При отсутствии признаковой памяти в контроллере делать по каждому приему строки несостоятельными все строки кэш-памяти.

Применение явной когерентности в создаваемых вычислительных системах обусловлено либо слишком большим временем, либо недопустимо высокими аппаратными затратами на реализацию неявного механизма когерентности.

Причина этого состоит в высокой сложности применения внутреннего механизма когерентности иерархической памяти узла по сравнению с

использованием предусмотренного для работы с внешними устройствами механизма когерентности.

Как правило, элементами разделяемой памяти при программной реализации когерентности являются страницы, что обусловлено использованием аппаратных средств организации виртуальной памяти для обнаружения операций записи в разделяемые страницы.

Однако относительно большой размер разделяемых страниц создает проблему ложного разделения, при котором конфликтными признаются модификации разными процессорами различных ячеек памяти одной страницы, не конфликтующие между собой. Решением данной проблемы может быть либо введение логических разделяемых переменных, объектов или типов данных, либо отображение множества логических страниц на одну физическую.

Механизм когерентности на основе масштабируемого когерентного интерфейса SCI

Масштабируемый когерентный интерфейс (Scalable Coheret Interface) принят в качестве стандарта ANSI. SCI предусматривает реализацию когерентности на основе стандартно организованной кэш-памяти дополнительного уровня, размещаемой в узле вычислительной системы.

Поиск данных в кэше узла производится при промахе в предыдущих уровнях иерархии памяти. При наличии данных в кэше узла и их состоятельности копия данных пересылается в верхние уровни памяти. Если данные находятся в кэше узла в модифицируемом состоянии, то ожидается окончание модификации, после чего копия данных перемещается выше по иерархии.

При отсутствии данных в кэше узла вырабатывается сигнал промаха, который активизирует контроллер интерфейса на действия по доставке данных из удаленных блоков памяти. Реализация узлов на основе SCI может поддерживать не все возможности SCI (не использовать кэш-памяти и протокола когерентности). В этом случае узлы SCI используются для построения коммуникационной среды со стандартным способом адресации, форматом и протоколом передачи пакетов.

Стандарт SCI позволяет строить коммуникационные среды для объединения узлов в сосредоточенные и распределенные вычислительные системы с быстродействием от 1 Гбит/с до 1 Гбайт/с и более.

Коммуникационная среда SCI предусматривает в каждом узле наличие входного и выходного каналов. Узлы связываются между собой или с коммутатором однонаправленными каналами "точка-точка". При объединении узлов должна формироваться циклическая магистраль (кольцо). Один из узлов в кольце выполняет функции инициализации узлов Кольца с установлением адресов, управления таймерами, уничтожения пакетов, не нашедших адресата. Данный узел, называемый scrubber и являющийся единственным в кольце, помечает проходящие через него пакеты и уничтожает уже помеченные. С помощью SCI могут быть реализованы различные структуры меж соединений: кольцо, тор, решетка.

Информационный обмен между узлами SCI реализуется с помощью транзакций чтения, записи, блокировки, когерентности строк кэш-памяти и основной памяти, передачи сообщений и сигналов прерываний. Все транзакции используют пересылку SCI пакетов между узлом-источником и узлом-получателем. Транзакции предусматривают передачу пакетов с блоками данных фиксированной длины 64 и 256 байтов и переменной длины от 1 до 16 байтов. Фиксированная длина пакетов упрощает логику и увеличивает быстродействие интерфейсов.

Интерфейсом SCI предусматривается 64-разрядная архитектура с 64-битным адресным пространством. Старшие 16 бит определяют адрес узла и применяются для маршрутизации SCI пакетов между узлами вычислительной системы. Остальные 48 битов адресуют внутреннюю память узла. Стандартом SCI предусматривается поддержка отображения и обмена между уровнями памяти блоками по 64 байта.

Протокол когерентности кэш-памятей определяет правила работы с множеством локальных копий одних и тех же данных. При аппаратной реализации этой функциональной возможности SCI снижается сложность операционной системы и системы программирования. В SCI протокол когерентности основан на распределенных директориях (каталогах). Все узлы с копиями данных участвуют в распределенном списке узлов, использующих одну и ту же строку данных. Признак каждой кэш строки содержит указатели на предыдущий и следующий узлы, разделяющие эту строку (имеющие копии строки в своей кэш-памяти). **Головной узел** содержит указатель на контроллер банка памяти, содержащего разделяемую строку, который, в свою очередь, имеет указатель на головной узел. **Оконечный узел** не содержит указателя на последующий узел. При этом каждая разделяемая строка памяти связана с распределенным списком узлов,

разделяющих эту строку. Все узлы, разделяющие строку, участвуют в формировании этого списка.

В зависимости от архитектуры процессора и исполняемой прикладной программы различают

- **когерентное кэширование,**
- **некогерентное кэширование,**
- **не кэширование строк памяти.**

При когерентном обращении узла к памяти за строкой контроллер памяти сохраняет адрес этого узла и возвращает адрес предыдущего узла, запрашивавшего эту строку. Если данные в памяти состоятельные (никакой узел не запрашивал доступа для изменения содержимого строки), то данные из памяти сразу передаются запросившему их узлу. Если строка памяти не состоятельна, а состоятельные данные находятся в каком-либо кэше, то возвращается только адрес узла, запрашивавшего эту строку последним. С помощью данного адреса запрашивающий узел обращается непосредственно в узел, имеющий состоятельные данные, после чего завершается формирование списка узлов, разделяющих строку.

Согласно протоколу доступ по записи в строку памяти представлен только узлу в начале списка. Для получения доступа по записи узел, бывший элементом списка, удаляет себя из списка и запрашивает доступ к памяти с правом записывать данные. В результате узел становится в начале списка и получает право модифицировать строку, но при этом обязан отметить как несостоятельные (очистить) все последующие элементы списка.

Стандартом предусмотрена сильная и слабая поддержка последовательного выполнения кода программ. При **слабой** поддержке операция записи в строку может быть произведена прежде, чем завершится очистка списка. При **сильной** поддержке последовательного выполнения программного кода запись должна производиться только после очистки списка. SCI поддерживает и линейное, и древовидное распространение очистки списка.

Предусматривается также возможность обмена двум узлам данными, находящимися в кэше, без обращения к памяти до тех пор, пока некий третий узел не запросит эти данные.

Основное достоинство протокола когерентности, используемого в SCI, состоит в том, что при расширении системы не нужно ничего менять в уже существующем вычислителе.

Недостатком является необходимость дополнительной памяти для хранения указателей списка.

Структура узла SCI предусматривает одновременную возможность отсылки сформированных в узле пакетов, приема пакетов, адресованных узлу, и ретрансляции транзитных пакетов. Для реализации указанных функций в узле имеется **три соответствующих FIFO очереди: выходная, входная и проходная**. Управление передачей пакетов между узлом-получателем и узлом-отправителем осуществляется в соответствии с протоколом, основанном на процедуре с решающей обратной связью. Размер окна управляющей процедуры равен 64.